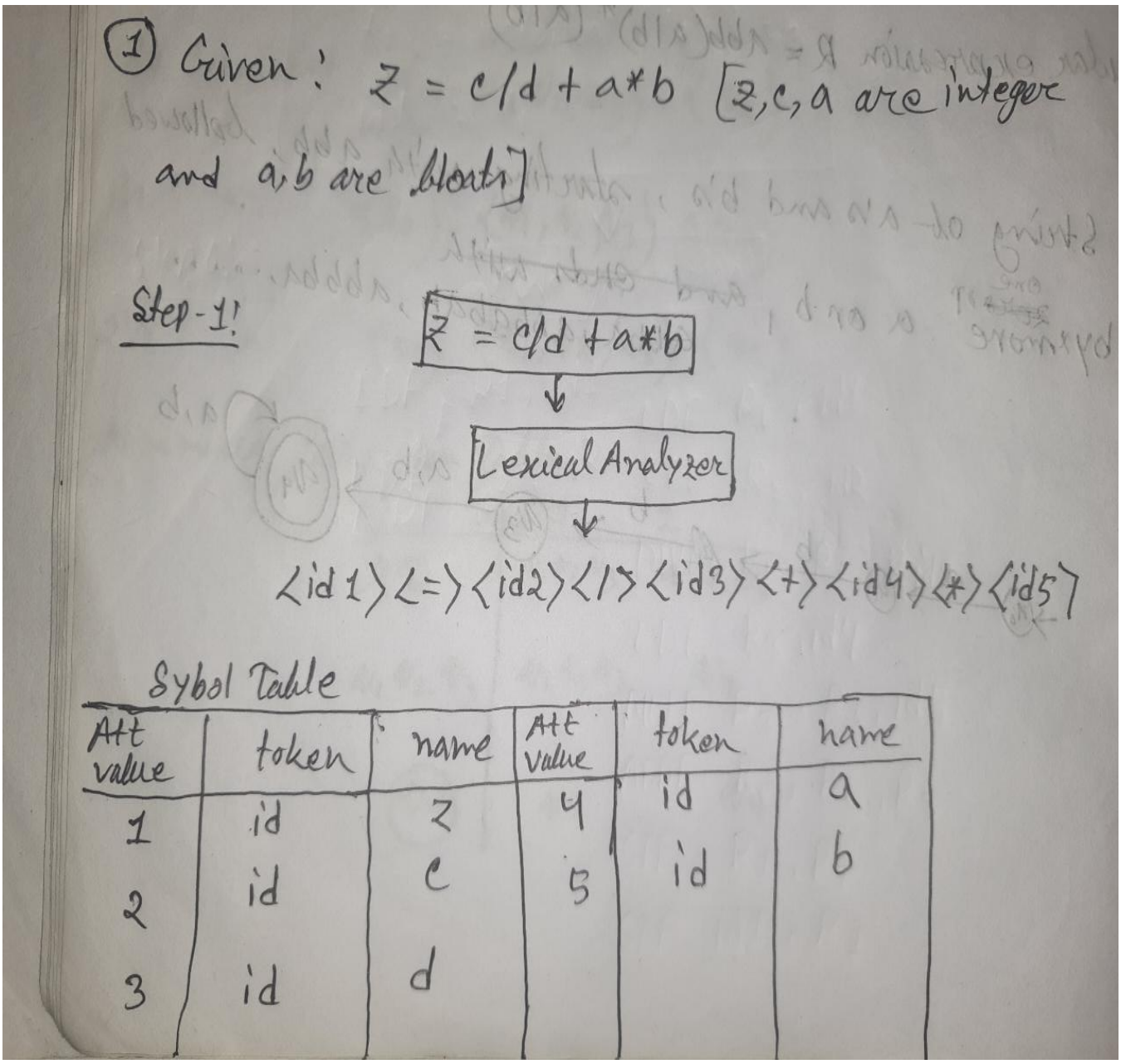
1(a)



(1) Given: $z = c/d + a*b$ [z, c, a are integer and a, b are float]

Step-1:

$z = c/d + a*b$

↓

Lexical Analyzer

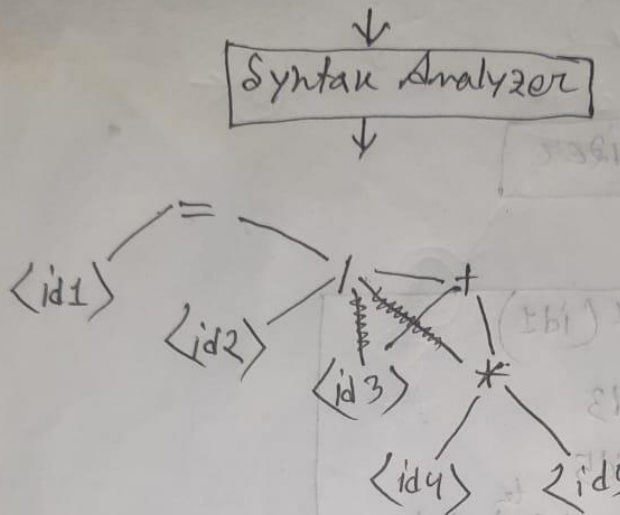
↓

<id 1><=><id2></><id3><+><id4><*><id5>

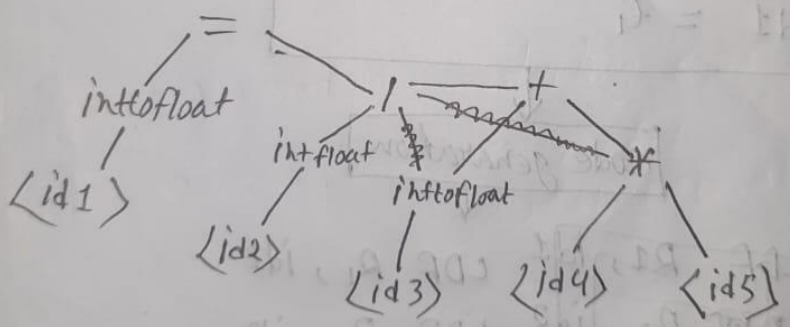
Sybol Table

Att value	token	name	Att value	token	name
1	id	z	4	id	a
2	id	c	5	id	b
3	id	d			

Step 2: output of lexical analyzer work as input of syntax analyzer.



Step 3:



Step 4:

Intermidiate code

t_1 = inttofloat (id1)
t_2 = inttofloat (id2)
t_3 = inttofloat (id3)
t_4 = t_2 / t_3
t_5 = id4 * id5

$t_6 = t_4 + t_5$

$id1 = t_6$

↓

Step 5: **Code Optimizer**

↓

$t_1 = \text{inttofloat}(id1)$

$t_2 = id2 / id3$

$t_3 = id4 * id5$

~~$t_1 = \text{inttofloat}(t_2) + t_3$~~

$t_1 = \text{inttofloat}(t_2) + t_3$

$id1 = t_1$

↓

Step 6: **code generation**

↓

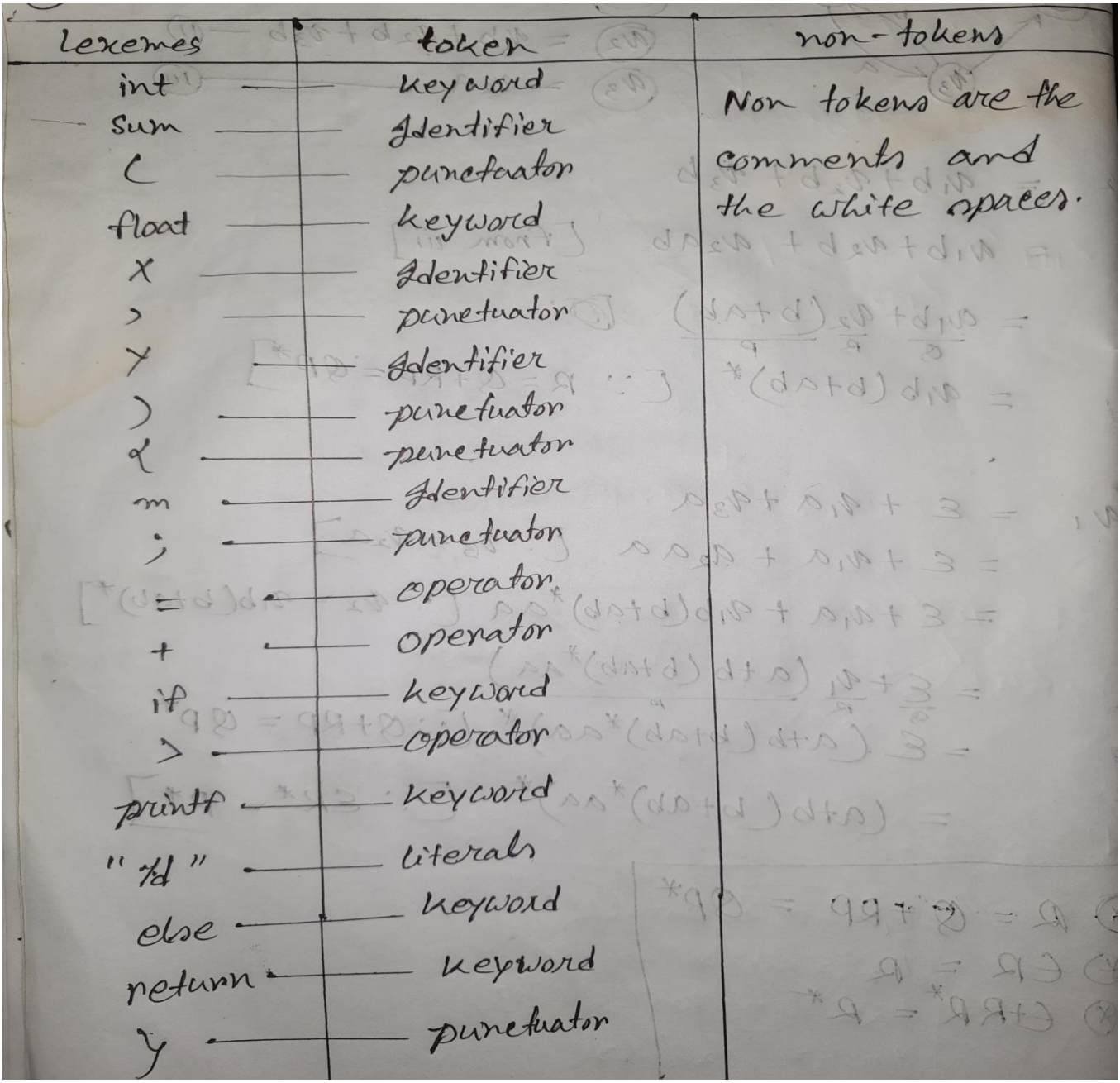
```
LDF  R1, id1          LDF  R1, id1
LDF  R2, id2          LDF  R2, id2
LDF  R3               DIVF R2, R2, id3
                      LDF  R3, id4
                      MULF R3, R3, id5
                      ADDF R2, R2, R3
                      STF  R1, R2
                      STF  id1, R1
```

1(b)

Lexical analyzer: It is the first phase of a compiler. The lexical analyzer reads the stream of characters making up the source program and groups the characters into meaningful sequences called lexemes. For each lexeme, the lexical analyzer produces as output a token of the form that it passes on to the subsequent phase, syntax analysis

Parser: It is the second phase of the compiler. The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream. A typical representation is a syntax tree in which each interior node represents an operation and the children of the node represent the arguments of the operation.

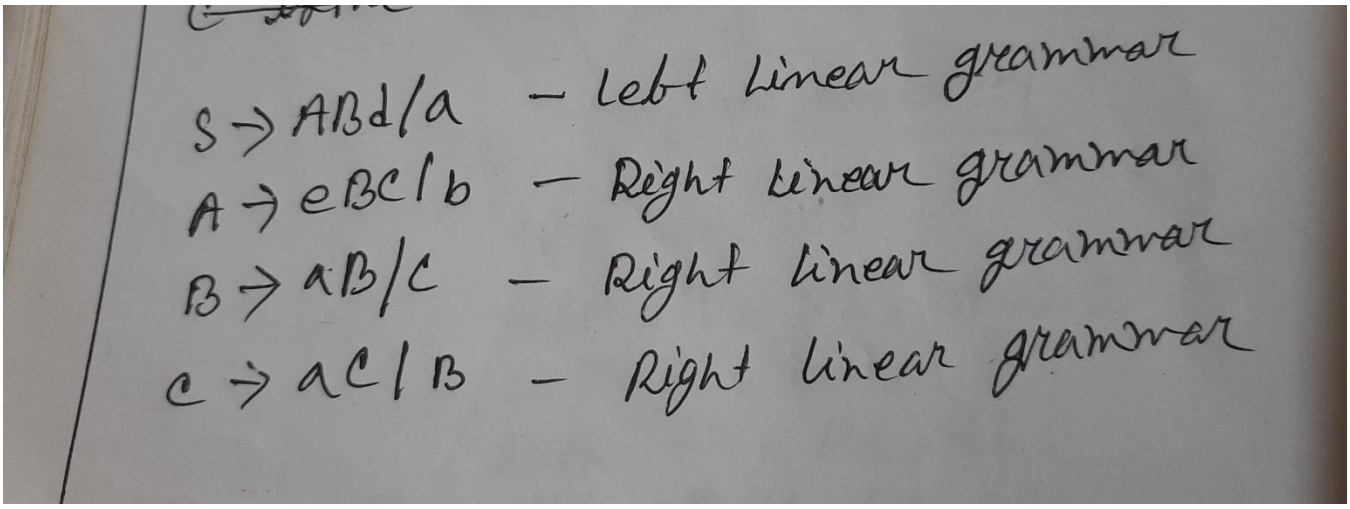
Lexemes, tokens and non-tokens of the code:



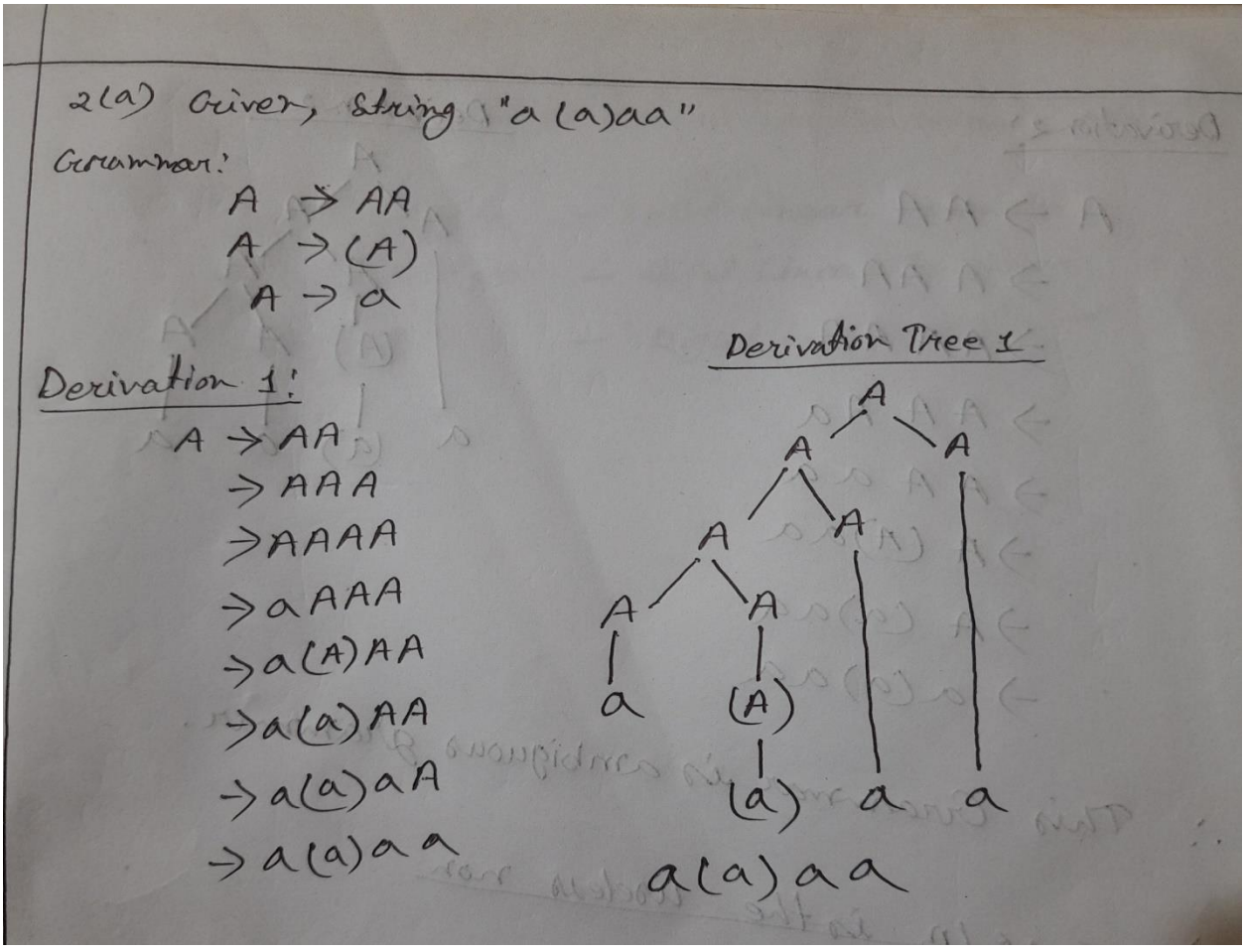
Lexemes	token	non-tokens
int	keyword	Non tokens are the comments and the white spaces.
sum	Identifier	
(	punctuator	
float	keyword	
x	Identifier	
>	punctuator	
y	Identifier	
)	punctuator	
{	punctuator	
m	Identifier	
;	punctuator	
=	operator	
+	operator	
if	keyword	
>	operator	
printf	keyword	
"%d"	literals	
else	keyword	
return	keyword	
}	punctuator	

2(a) A context-sensitive grammar (CSG) is a formal grammar in which the left-hand sides and right-hand sides of any production rules may be surrounded by a context of terminal and nonterminal symbols.

C is the useless non-terminal here.



2(a) OR: If a context free grammar G has more than one derivation tree for some string w ∈ L(G), it is called an ambiguous grammar. There exist multiple right-most or leftmost derivations for some string generated from that grammar.



Derivation 2:

$A \rightarrow AA$

$\rightarrow A\,AA$

$\rightarrow AA\,AA$

$\rightarrow AA\,Aa$

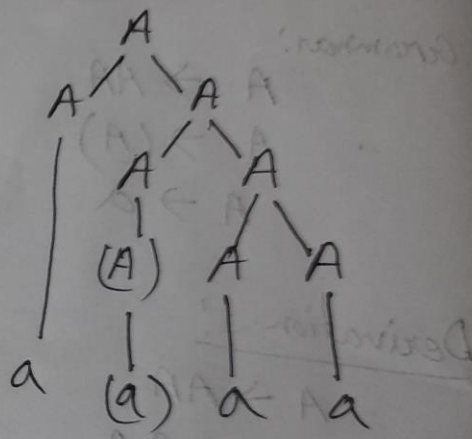
$\rightarrow AA\,aa$

$\rightarrow A\,(A)aa$

$\rightarrow A\,(a)aa$

$\rightarrow a(a)aa$

Derivation 2



∴ This Grammar is ambiguous grammar.

2(b)

(*) CFG for the language of all non palindromes.

$A \rightarrow aAa \mid bAb \mid aBb \mid bBa$

$B \rightarrow aB \mid bB \mid \varepsilon$

OR

(*) Given $L = 0^n 1^{4n}$ where $n >= 1$

$= \{01111, 00\,11111111, \ldots\ldots\}$

CFG: $S \rightarrow 0A$

$A \rightarrow 1111A \mid S \mid \varepsilon$

2(c)

(c) i) All strings of a's and b's with an even number of a's and odd numbers of b's.

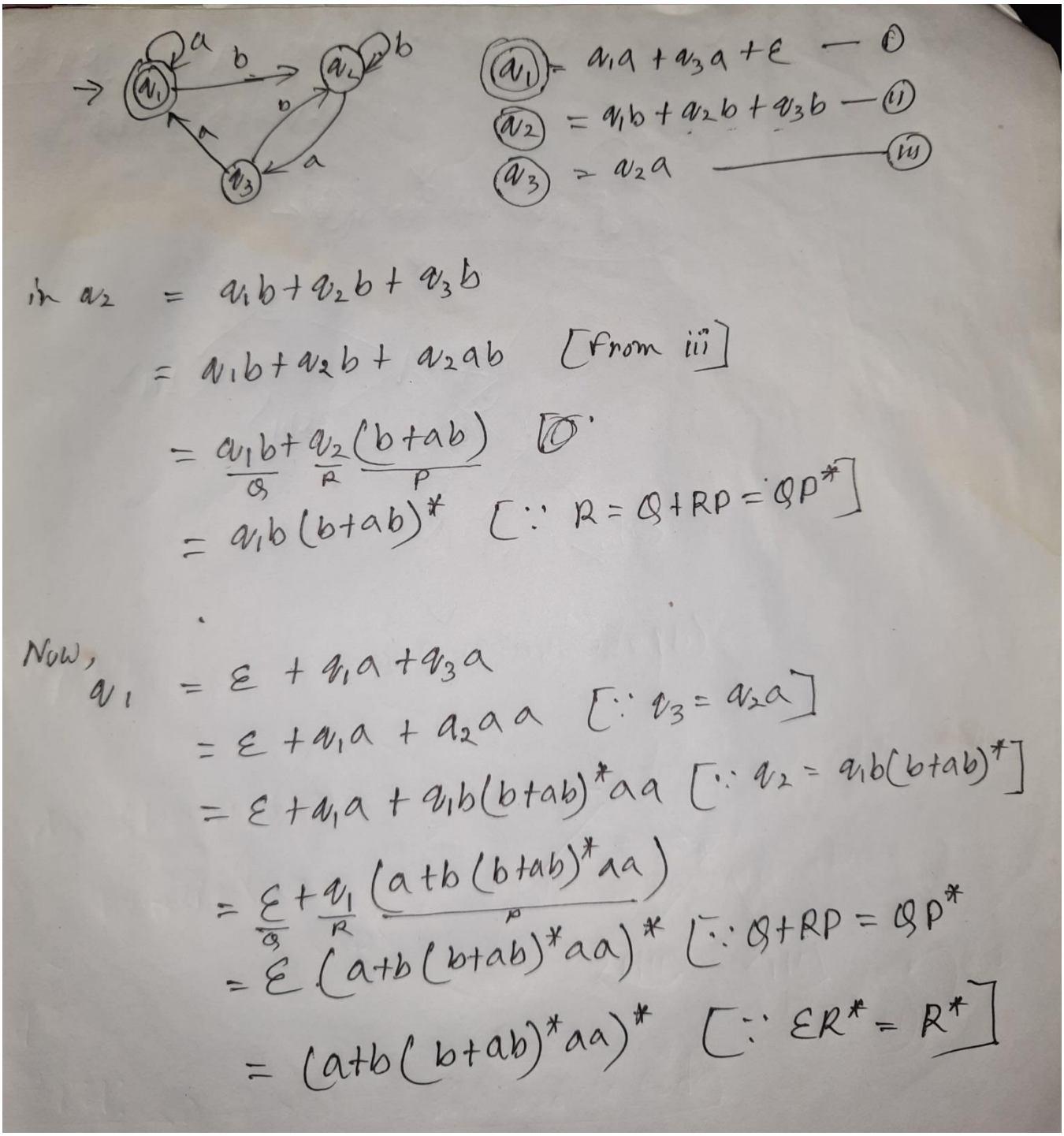
$L = \{aab, baa, aaaab, baaaa, aabbb, bbbaa \ldots\ldots\}$

Regular Expression = $(aa)^*(bb)^*b$

ii) All strings of a's and b's that do not contain the substring abb.

Regular Expression = ~~b^*a^*~~ $b^*(a(\epsilon \mid b))^*$

3(a)


a
b
b
a
b
a
a
q_1
q_2
q_3
$q_1 = q_1a + q_3a + \varepsilon$ — (i)
$q_2 = q_1b + q_2b + q_3b$ — (ii)
$q_3 = q_2a$ — (iii)
in q_2 $= q_1b + q_2b + q_3b$
$= q_1b + q_2b + q_2ab$ [From iii]
$= \frac{q_1b}{Q} + \frac{q_2}{R}\frac{(b+ab)}{P}$
$= q_1b(b+ab)^*$ [$\because R = Q + RP = QP^*$]
Now, $q_1 = \varepsilon + q_1a + q_3a$
$= \varepsilon + q_1a + q_2aa$ [$\because q_3 = q_2a$]
$= \varepsilon + q_1a + q_1b(b+ab)^*aa$ [$\because q_2 = q_1b(b+ab)^*$]
$= \frac{\varepsilon}{Q} + \frac{q_1}{R}\frac{(a+b(b+ab)^*aa)}{P}$
$= \varepsilon(a+b(b+ab)^*aa)^*$ [$\because Q + RP = QP^*$
$= (a+b(b+ab)^*aa)^*$ [$\because \varepsilon R^* = R^*$]

3(b)

3)b) Minimizing the DFA:

Transition Table for DFA:

	0	1
→ q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_1	q_4
q_3	q_2	q_4
(q_4)	q_4	q_4

0's compliment $\{q_0\ q_1\ q_2\ q_3\}$ $\{q_4\}$

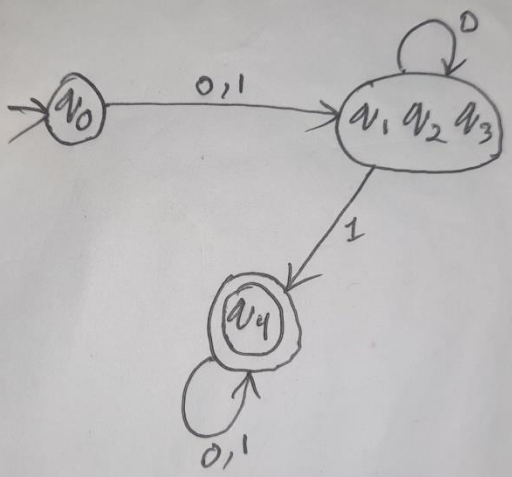
1's " $\{q_0\}$ $\{q_1, q_2, q_3\}$ $\{q_4\}$

2's " $\{q_0\}$ $\{q_1, q_2, q_3\}$ $\{q_4\}$

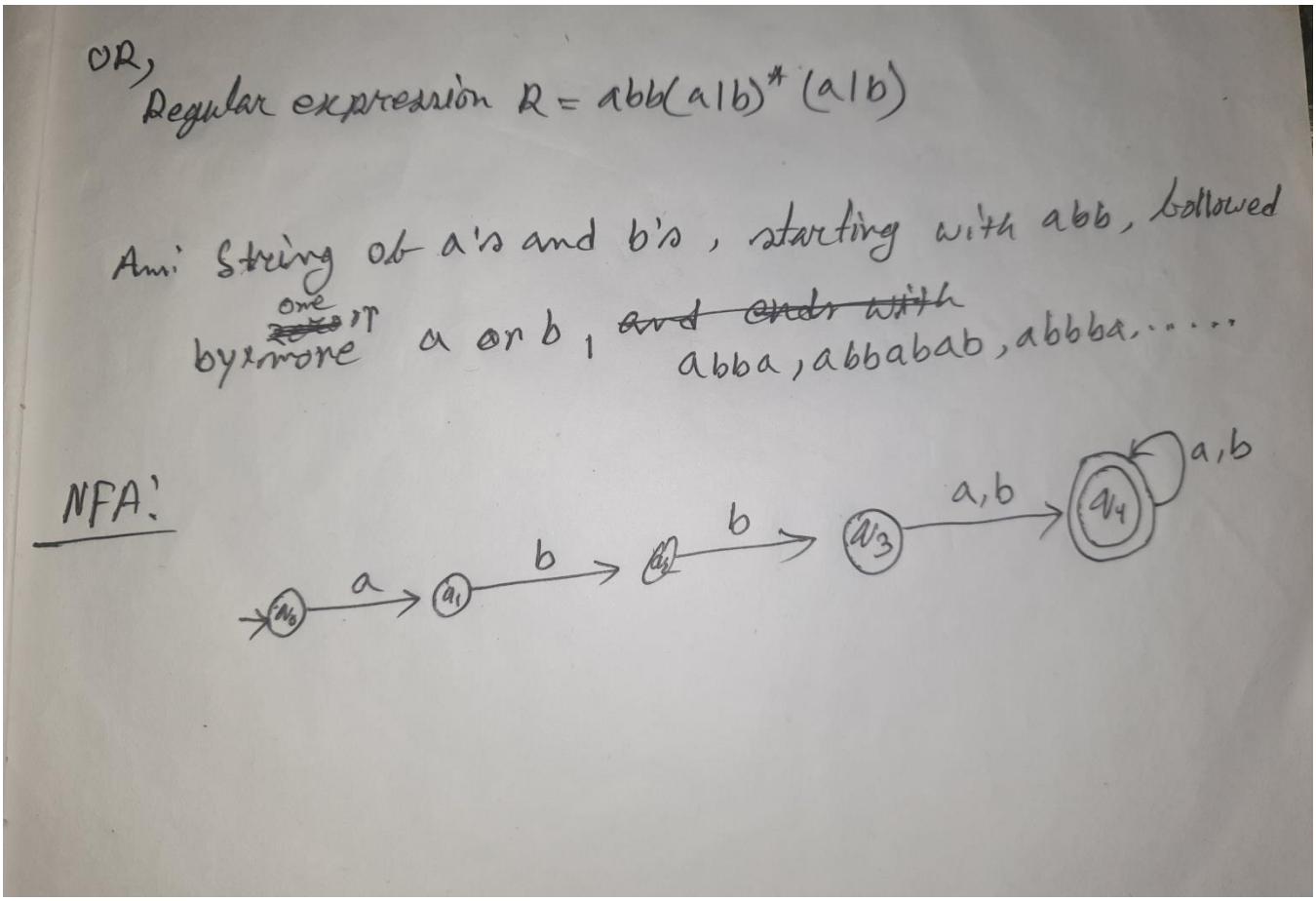
Minimized Transition Table:

	0	1
→ q_0	$q_1q_2q_3$	$q_1q_2q_3$
$q_1q_2q_3$	$q_1q_2q_3$	q_4
(q_4)	q_4	q_4

∴ Minimized DFA:



OR



3(c)

i) a(a | b)*a

Answer: String of a`s and b`s begin and end with a.

ii) a*ba*ba*ba*

Answer: String of a`s and b`s containing 3 b’s.